

PYTHON FILE HANDLING & AUTOMATION

Project Report

Student Name: Bammidi Rakesh

Roll Number: 23981A04Q9

Department: ECE

Institution: Raghu Engineering College

Date: 7 September 2026

1. Introduction

Python provides built-in modules and functions for working with files and folders. This project implements a menu-driven Automated File Manager that demonstrates text-file handling, CSV processing, file renaming, moving, deletion, and exception handling.

2. Objectives

- Create and read TXT files.
- Create and read CSV files using Python's csv module.
- Automate common file operations such as rename, move, and delete.
- Use try-except blocks to handle runtime errors safely.
- Build a simple menu-driven command-line application.

3. Tools and Technologies

- Programming Language: Python 3
- IDE: Visual Studio Code
- Modules: os, csv, shutil
- Operating environment: Windows

4. Project Structure

```
Python_File_Automation/  
■■■■ file_manager.py  
■■■■ input_files/  
■■■■ output_files/  
■■■■ screenshots/
```

5. Functional Description

Option 1 creates a TXT file in input_files. Option 2 reads and displays TXT content. Option 3 creates a CSV file with Name, Age, and Department fields. Option 4 reads the CSV row by row. Option 5 renames a file, Option 6 moves a file from input_files to output_files, and Option 7 deletes a file after confirmation. Option 8 terminates the program.

6. Exception Handling

The project uses try-except blocks to prevent the program from terminating unexpectedly. FileNotFoundError is handled specifically while reading files, and other exceptions are reported with useful error messages. Rename operations also handle FileExistsError.

7. Complete Python Source Code

```
import os  
import csv  
import shutil  
  
input_folder = "input_files"  
output_folder = "output_files"  
  
os.makedirs(input_folder, exist_ok=True)  
os.makedirs(output_folder, exist_ok=True)  
  
# -----  
# MAIN MENU  
# -----  
  
while True:  
  
    print("\n=====")  
    print("      AUTOMATED FILE MANAGER")  
    print("=====")  
  
    print("1. Create TXT File")  
    print("2. Read TXT File")  
    print("3. Create CSV File")  
    print("4. Read CSV File")  
    print("5. Rename File")  
    print("6. Move File")  
    print("7. Delete File")  
    print("8. Exit")  
  
    choice = input("\nEnter your choice: ")  
  
    if choice == "1":  
        try:
```

```

        file_name = input("Enter TXT file name: ")
        file_path = os.path.join(input_folder, file_name + ".txt")
        content = input("Enter the content to write: ")
        with open(file_path, "w") as file:
            file.write(content)
        print("\nTXT file created successfully.")
        print("File location:", file_path)
    except Exception as error:
        print("Error while creating TXT file:", error)

elif choice == "2":
    try:
        file_name = input("Enter TXT file name: ")
        file_path = os.path.join(input_folder, file_name + ".txt")
        with open(file_path, "r") as file:
            content = file.read()
        print("\n----- FILE CONTENT -----")
        print(content)
        print("-----")
    except FileNotFoundError:
        print("Error: TXT file not found.")
    except Exception as error:
        print("Error while reading TXT file:", error)

elif choice == "3":
    try:
        csv_name = input("Enter CSV file name: ")
        csv_path = os.path.join(input_folder, csv_name + ".csv")
        with open(csv_path, "w", newline="") as file:
            writer = csv.writer(file)
            writer.writerow(["Name", "Age", "Department"])
            print("\nEnter student details.")
            name = input("Enter name: ")
            age = input("Enter age: ")
            department = input("Enter department: ")
            writer.writerow([name, age, department])
        print("\nCSV file created successfully.")
        print("File location:", csv_path)
    except Exception as error:
        print("Error while creating CSV file:", error)

elif choice == "4":
    try:
        csv_name = input("Enter CSV file name: ")
        csv_path = os.path.join(input_folder, csv_name + ".csv")
        with open(csv_path, "r") as file:
            reader = csv.reader(file)
            print("\n----- CSV CONTENT -----")
            for row in reader:
                print(" | ".join(row))
            print("-----")
    except FileNotFoundError:
        print("Error: CSV file not found.")
    except Exception as error:
        print("Error while reading CSV file:", error)

elif choice == "5":
    try:
        old_file = input("Enter current file name: ")
        new_file = input("Enter new file name: ")
        old_path = os.path.join(input_folder, old_file)
        new_path = os.path.join(input_folder, new_file)
        if os.path.exists(old_path):
            os.rename(old_path, new_path)
            print("\nFile renamed successfully.")
            print("New file name:", new_file)
        else:
            print("Error: File not found.")
    except FileExistsError:
        print("Error: A file with the new name already exists.")
    except Exception as error:
        print("Error while renaming file:", error)

elif choice == "6":
    try:
        file_name = input("Enter file name to move: ")
        source_path = os.path.join(input_folder, file_name)
        destination_path = os.path.join(output_folder, file_name)
        if os.path.exists(source_path):
            shutil.move(source_path, destination_path)
            print("\nFile moved successfully.")
            print("Moved to:", destination_path)
        else:
            print("Error: File not found.")
    except Exception as error:
        print("Error while moving file:", error)

elif choice == "7":
    try:
        file_name = input("Enter file name to delete: ")
        file_path = os.path.join(input_folder, file_name)
        if not os.path.exists(file_path):
            file_path = os.path.join(output_folder, file_name)
        if os.path.exists(file_path):
            confirmation = input(
                "Are you sure you want to delete this file? (y/n): "
            )
            if confirmation.lower() == "y":
                os.remove(file_path)
                print("File deleted successfully.")
            else:
                print("Delete operation cancelled.")
        else:
            print("Error: File not found.")
    except Exception as error:

```

```
        print("Error while deleting file:", error)

elif choice == "8":
    print("\nThank you for using Automated File Manager.")
    print("Exiting program...")
    break

else:
    print("\nInvalid choice. Please enter a number from 1 to 8.")
```

8. Output and Execution Evidence

The following screenshots document the successful execution of each major feature.

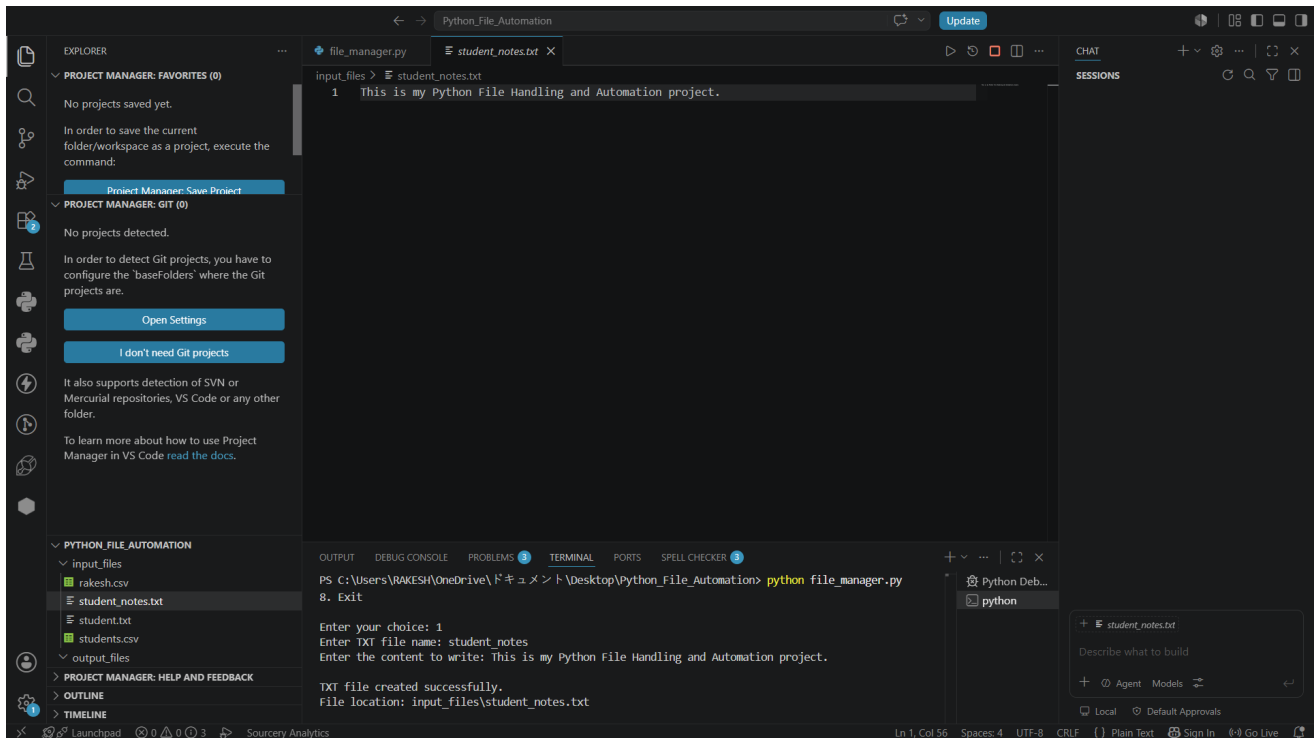


Figure 1 – Create TXT File

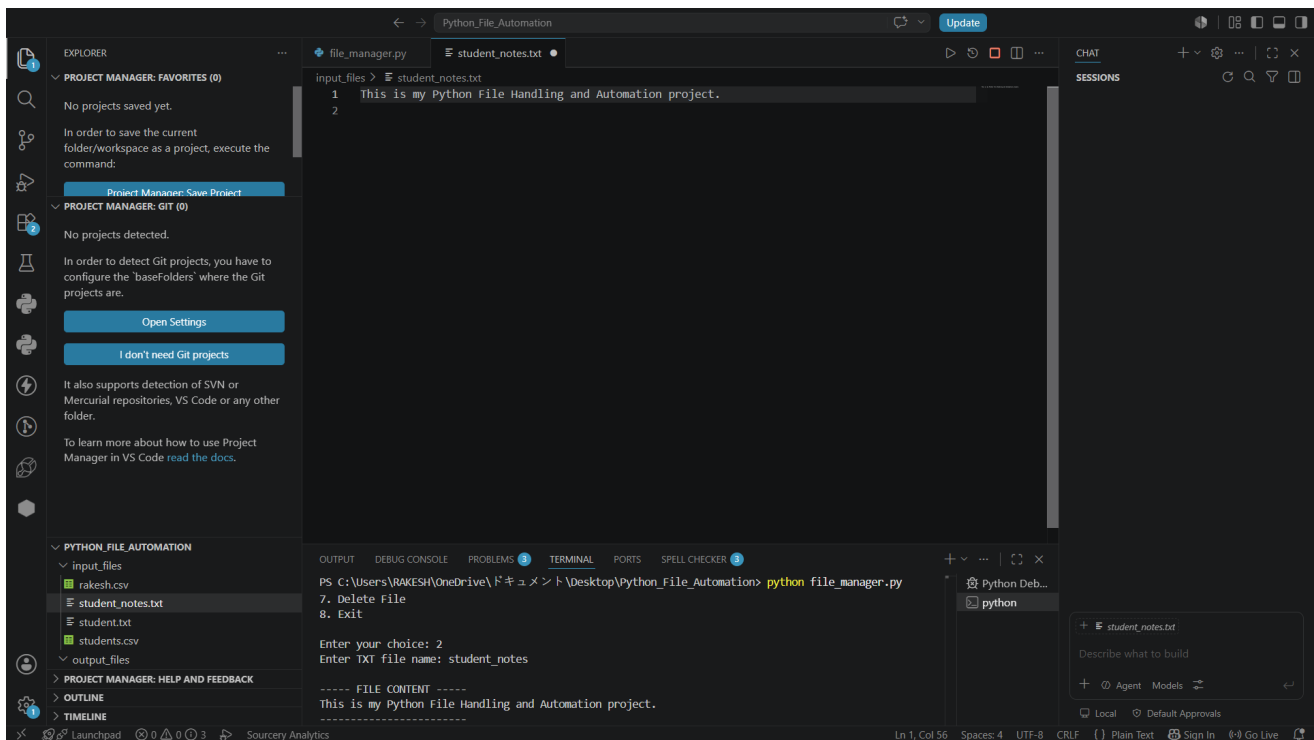


Figure 2 – Read TXT File

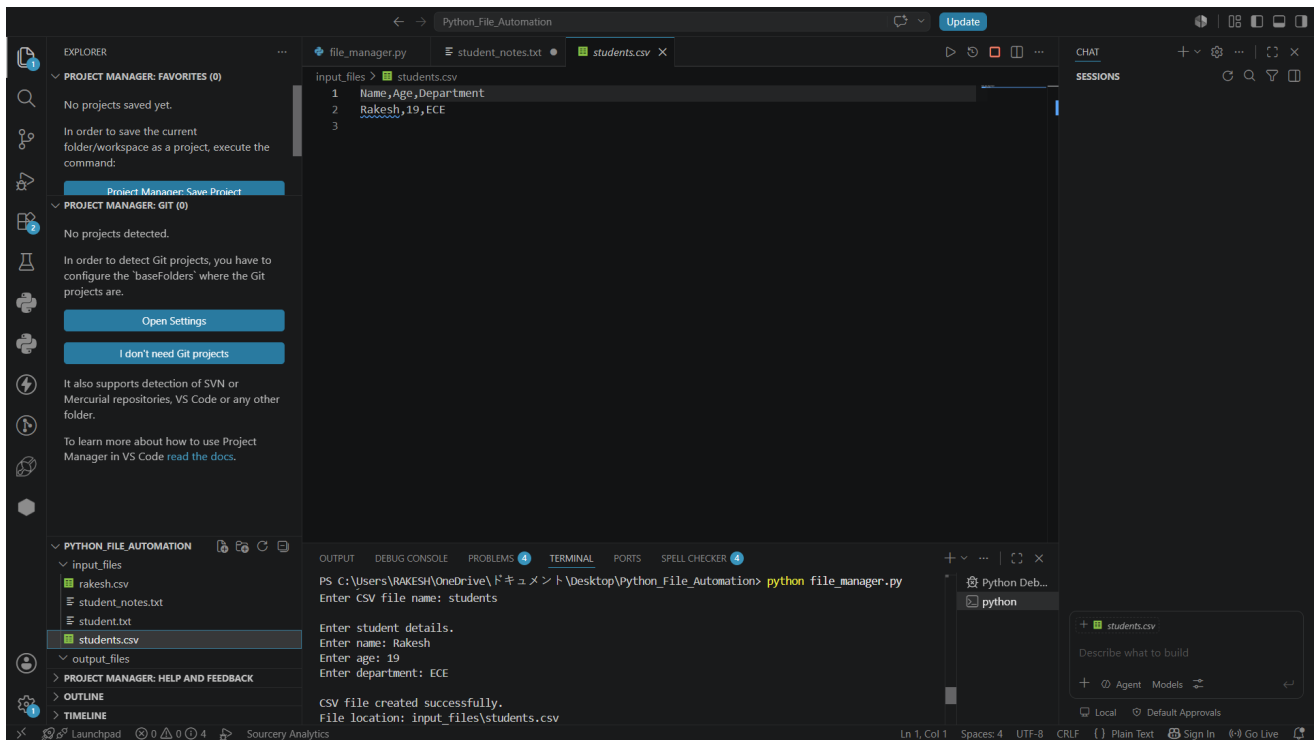


Figure 3 – Create CSV File

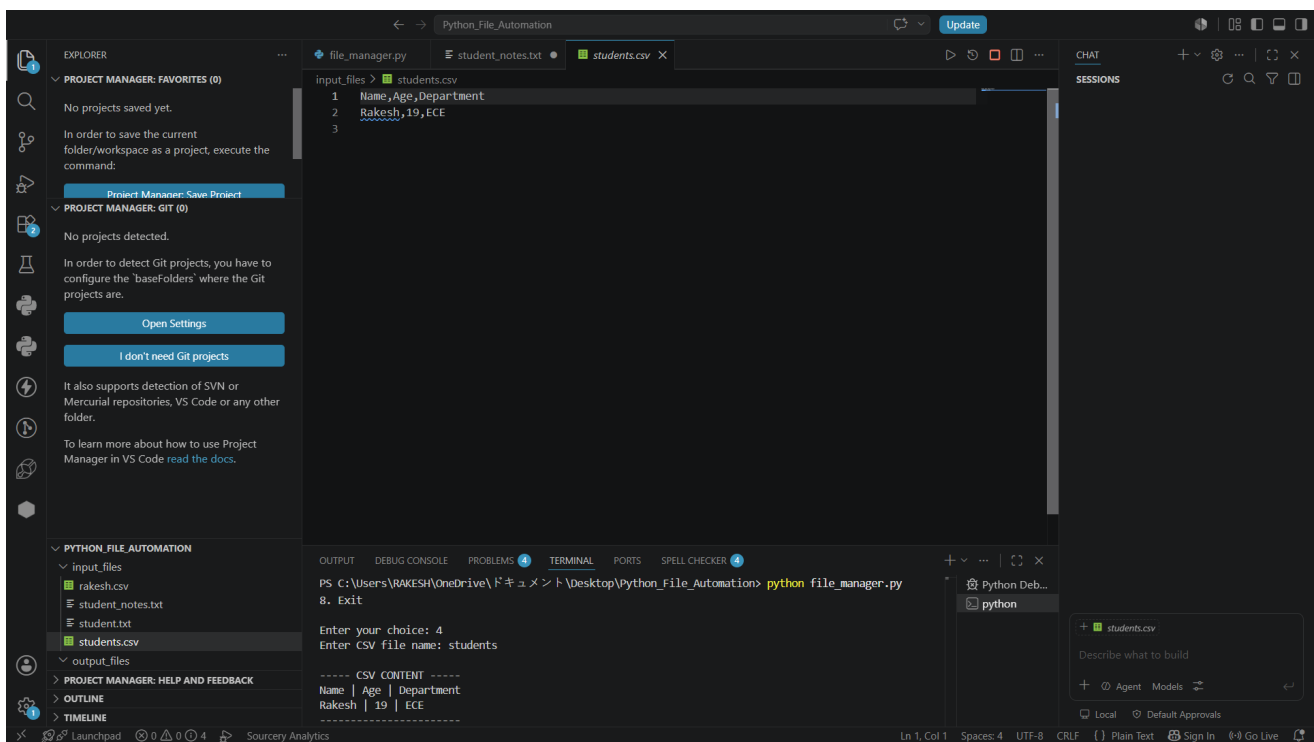


Figure 4 – Read CSV File

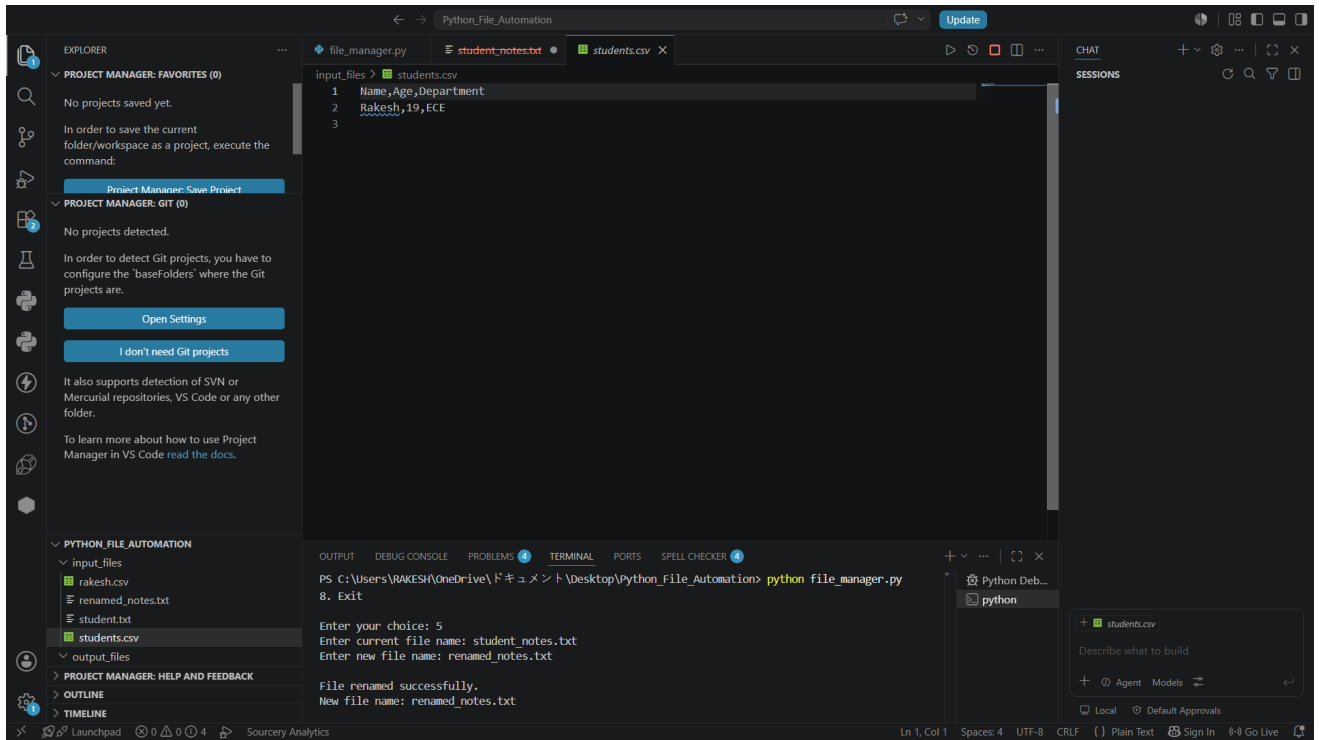


Figure 5 – Rename File

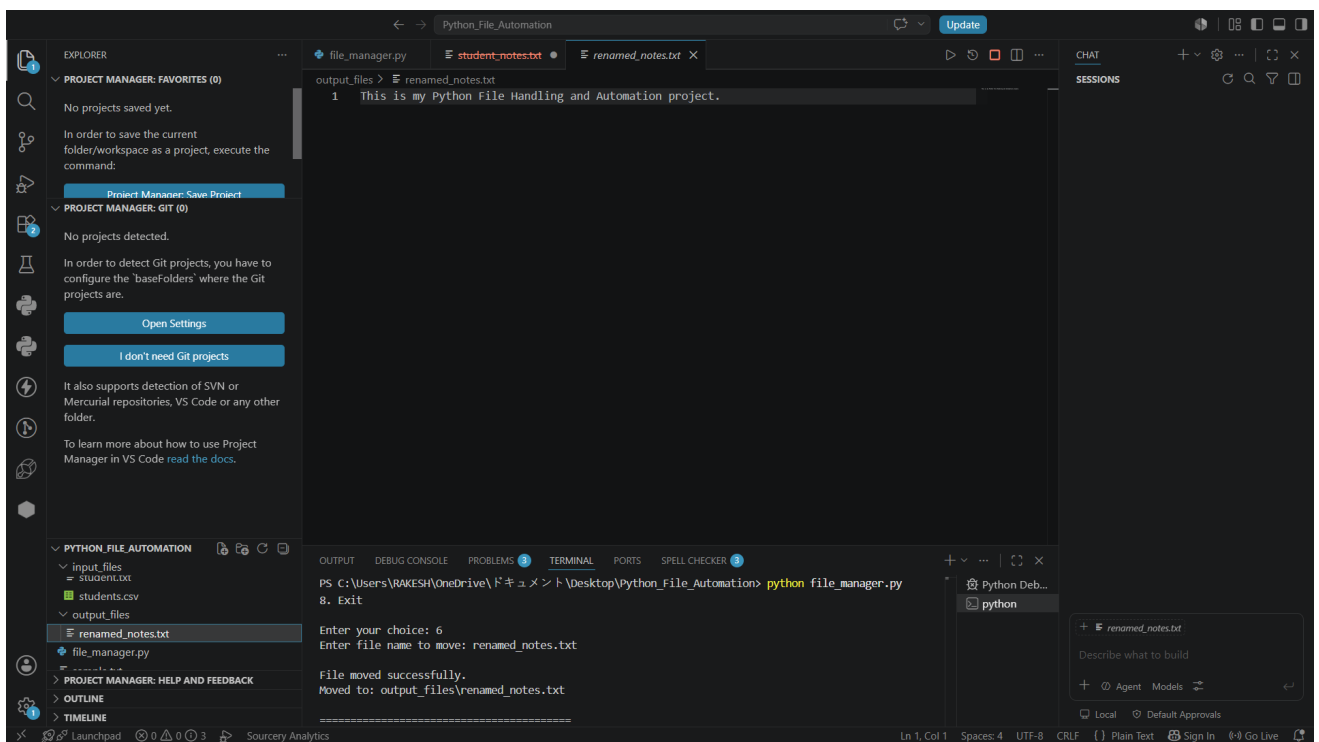


Figure 6 – Move File

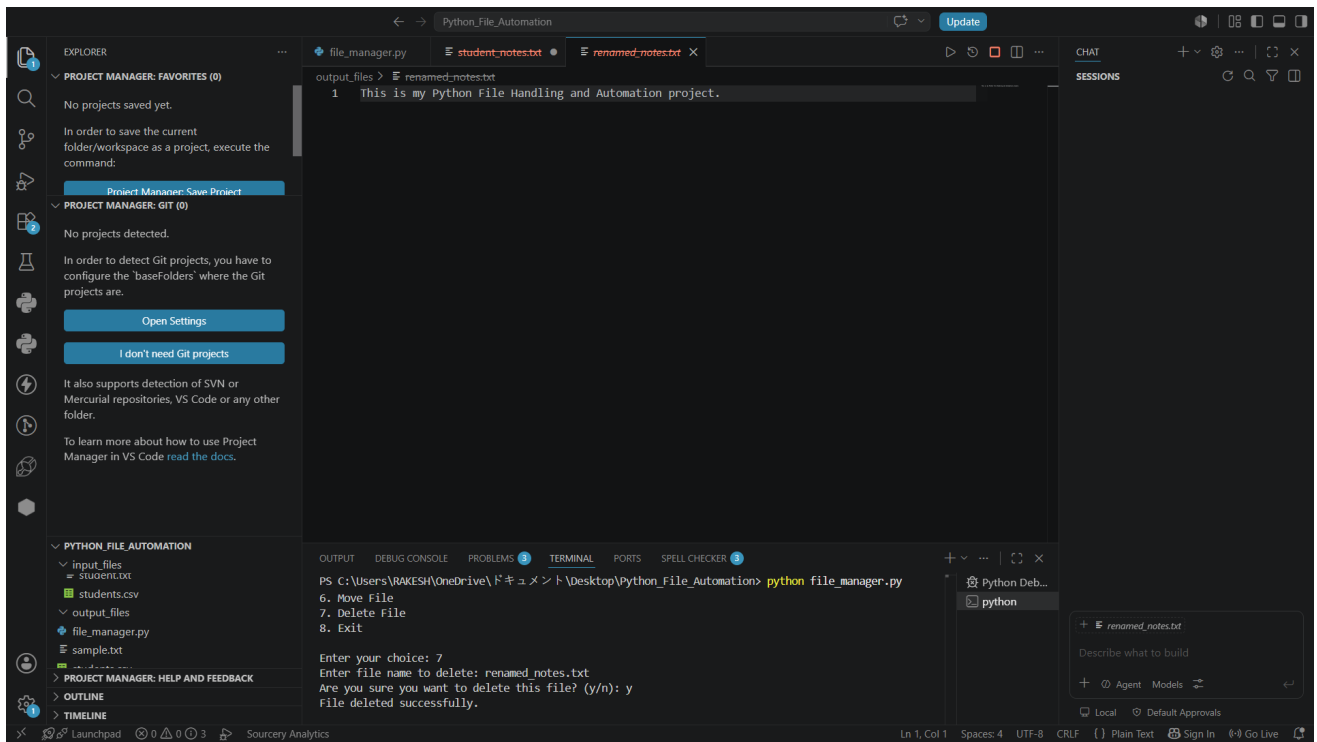


Figure 7 – Delete File

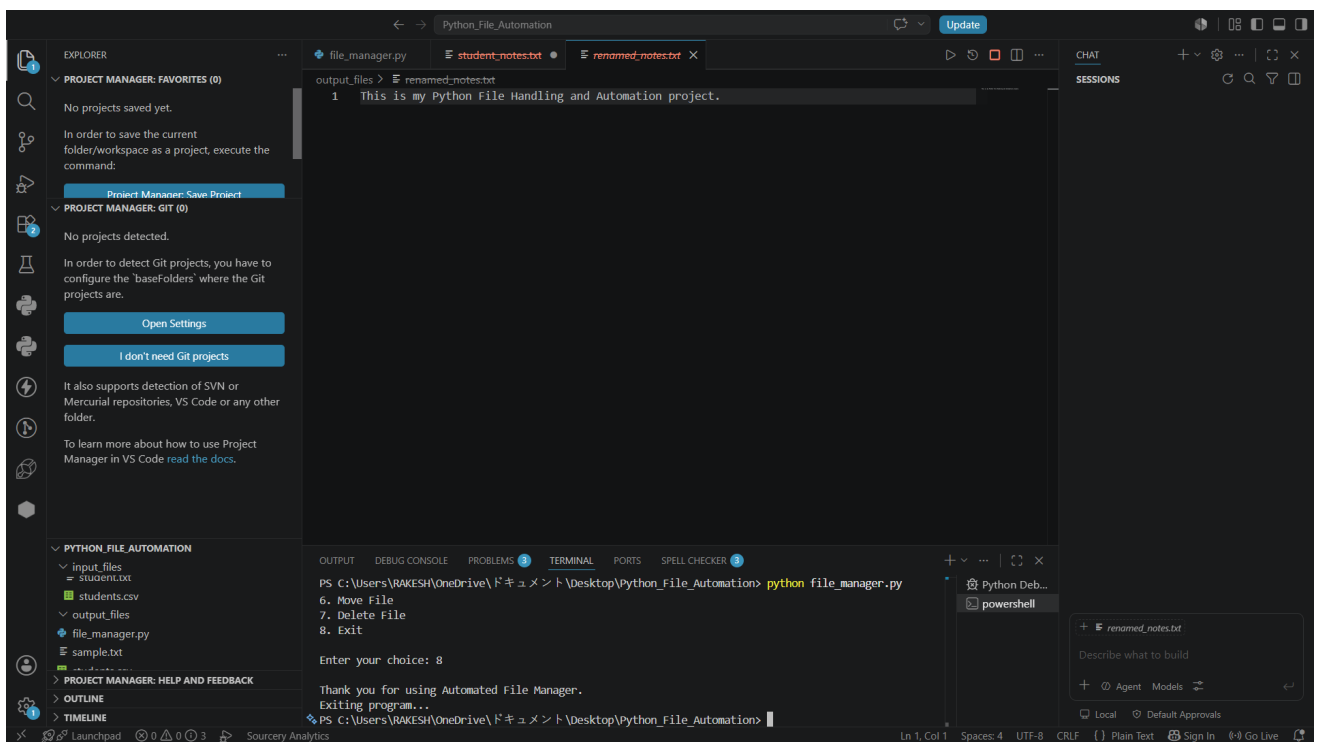


Figure 8 – Exit Program

9. Advantages

- Simple menu-driven interface.
- Supports TXT and CSV formats.
- Automates rename, move, and delete operations.
- Includes deletion confirmation.
- Uses exception handling for safer execution.

10. Applications

- File and document management.
- Python programming education.
- Small-scale automation.
- Organizing generated or processed files.

11. Conclusion

The Automated File Manager successfully demonstrates core Python file-handling and automation concepts. It combines TXT and CSV operations with practical file-system automation using `os` and `shutil`, while `try-except` blocks improve reliability.

12. Submission Link

Google Drive / GitHub Public Link: _____

Task Submission Page: _____

<https://github.com/bammidirakesh/Python-File-Automation>